

DESSINS DE GRAPHES AVEC **GRAPHVIZ**

(Pierre Fouilhoux, Pierre Rousselin, Antoine Rozenknop)

L'objectif de ce sujet de TP est de dessiner des graphes avec la commande `dot`. Nous en profiterons pour faire des petites révisions sur les entrées/sorties et manipuler des matrices d'adjacence.

1 Le langage DOT de la suite *graphviz*

La suite de logiciels *graphviz* a pour but la représentation graphique, dans divers formats (`pdf`, `png`, ...), de graphes orientés ou non en utilisant différents algorithmes.

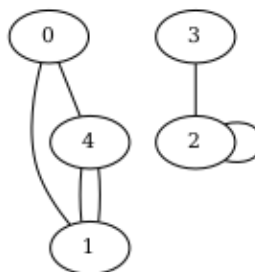
Pour notre part nous utiliserons la commande `dot`. Cette commande a la syntaxe suivante :

```
$ dot -Text fichier_source.dot -o fichier_dest.ext
```

où `ext` est l'extension associée au format voulu (par exemple `pdf`) et `fichier_source.dot` est un fichier texte écrit dans le langage DOT de descriptions de graphes. Voyons tout de suite un exemple de tel fichier et la sortie correspondante.

```
graph {
    0;
    1;
    2;
    3;
    4;

    0 -- 1;
    2 -- 2;
    3 -- 2;
    4 -- 1;
    0 -- 4;
    4 -- 1;
}
```



Dans cet exemple, le fichier DOT s'appelait `exemple1.dot` et le fichier `pdf` a été généré avec la commande

```
$ dot -Tpdf exemple1.dot -o mon_graphe.pdf
```

puis visualisé avec la commande

```
$ evince mon_graphe.pdf &
```

(le `&` final permet de mettre la commande en arrière-plan et donc de garder la main sur le terminal).

Pour se contenter d'afficher le graphe dans une fenêtre, sans écrire de fichier, on peut entrer la commande suivante :

\$ dot -Tx11 exemple1.dot

Dans le code DOT, le mot-clé **graph** sert à signaler qu'on considère un graphe *non-orienté*. Entre les accolades, le graphe est décrit¹. Les premières lignes, ne comportant qu'une seule étiquette, servent à définir les sommets du graphe, et les lignes suivantes (de la forme *i -- j*;) servent à décrire les arêtes du graphe.

Question 1: Décrire au format DOT le graphe représenté à la figure 1, puis visualiser avec une commande **dot** le graphe correspondant.

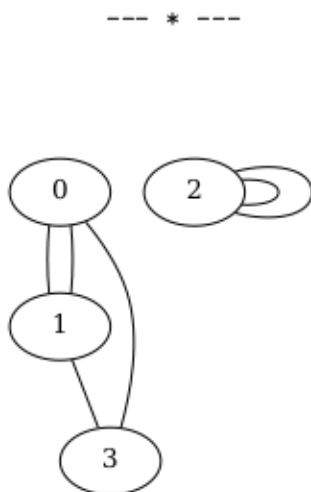


FIGURE 1 – Graphe à écrire dans la question 1.

Question 2: Écrire dans le fichier **complet-5.dot** le graphe complet à 5 sommets et visualiser le résultat avec une commande **dot**.

--- * ---

2 Écrire un fichier dot avec un programme en C

L'écriture directe de gros graphes au format **dot** peut être assez pénible. Le but est donc d'écrire, en langage C, des programmes qui écrivent eux-mêmes des fichiers **dot**. *Pour ceux qui auraient besoin d'un rafraîchissement sur les entrées/sorties en C, voir l'annexe en fin de sujet.*

Question 3: Écrire dans un fichier **complet-10.c** un programme simple en C qui décrit dans un fichier au format DOT appelé **complet-10.dot**, le graphe complet d'ordre 10.

Compiler et exécuter votre programme, puis dessiner ce graphe en utilisant une commande **dot**.

--- * ---

1. On peut être beaucoup plus concis en langage DOT mais le format très rigide que nous avons choisi est facilement automatisable.

3 De la matrice d'adjacence au fichier dot

Nous allons maintenant parler de matrices d'adjacence.

Question 4: En mode débranché! Écrire les matrices d'adjacence des graphes précédemment considérés (le premier exemple, celui de la figure 1 et le graphe complet d'ordre 5).

--- * ---

Pour ce TP introductif, nous allons simplement utiliser un tableau statique global²

```
int mat[ORDRE_MAX][ORDRE_MAX];
```

pour stocker notre matrice d'adjacence.

Il s'agit de compléter le programme `matrice_adj.c` fourni.

Dans toute les fonctions à compléter, le paramètre `int n` est l'ordre du graphe, avec comme pré-condition `n > 0 && n <= ORDRE_MAX`.

La fonction `afficher_mat` est déjà écrite. La lecture de son corps peut vous rafraîchir la mémoire.

Question 5: Compléter la fonction

```
void graphe_complet(int n);
```

qui met dans `mat` la matrice d'adjacence du graphe complet à `n` sommets.

Vérifier le résultat à l'aide de la fonction (déjà écrite) `afficher_mat`.

--- * ---

Question 6: Faire de même pour la fonction `graphe_stable` (un graphe stable est simplement un graphe *sans arête*).

--- * ---

Question 7: Écrire la fonction

```
int ecrire_dot(int n, const char *nom_fichier);
```

qui écrit dans le fichier `nom_fichier` au format `dot` le graphe d'ordre `n` dont la matrice d'adjacence est stockée dans `mat`. Respecter le format donné dans le premier exemple (espaces autour de `--`, ligne vide après l'écriture des sommets), cela simplifiera beaucoup l'écriture de la fonction `lire_dot`.

Tester en écrivant dans un fichier `dot` le graphe complet à 10 sommets et le graphe stable à 10 sommets. Visualiser le résultat.

--- * ---

2. L'utilisation de variables globales est souvent considéré comme une **mauvaise pratique**, mais ici il s'agit juste d'écrire un petit programme qui dessine des graphes et cela simplifie l'énoncé.

4 Quelques familles de graphes non orientés

Question 8: Écrire la fonction

```
void graphe_cycle(int n);
```

Notre graphe cycle à n sommets aura les arêtes $\{0, 1\}$, $\{1, 2\}$, ..., $\{n-2, n-1\}$ et $\{n-1, 0\}$.
Tester cette fonction en visualisant un graphe cycle, par exemple de 8 sommets.

--- * ---

Le graphe biparti complet $K_{m,p}$, où m et p sont des entiers positifs non nuls a $m+p$ sommets et pour ensemble d'arêtes

$$\left\{ \{i, j\} \mid 0 \leq i \leq m-1, m \leq j \leq m+p-1 \right\}.$$

Question 9: Dessiner à la main $K_{3,2}$, puis écrire la fonction

```
void graphe_biparti_complet(int m, int p);
```

Tester en visualisant, par exemple, $K_{5,3}$.

--- * ---

Un graphe d'Erdős-Rényi $G(n, p)$, où n est un entier positif non nul et p est un réel compris entre 0 et 1 est un graphe aléatoire simple à n sommets, où chaque arête est présente³ avec probabilité p (absente avec probabilité $1-p$).

Pour ceux qui ont besoin d'un rappel sur l'aléatoire en C, voici un petit programme qui simule 10 lancers (considérés comme indépendants) de pile ou face.

```
#include <stdio.h>
#include <stdlib.h> /* rand, srand, RAND_MAX */
#include <time.h> /* time pour initialiser la graine */

int main()
{
    int i;
    double U;
    /* Initialisation du générateur de nombre aléatoire :
     * la graine est donnée par le nombre de secondes depuis
     * le 1er janvier 1970, et sera donc différente à chaque exécution (ou
     * presque). */
    srand(time(NULL));
    /* 10 tirages de pile ou face (affichage sur le terminal) */
    for (i = 0; i < 10; ++i) {
        U = (double) rand() / RAND_MAX; /* U aléatoire entre 0 et 1 */
        if (U <= .5)
            printf("PILE !\n");
        else
```

3. Les événements « l'arête $\{i, j\}$ est présente », pour $0 \leq i < j \leq n-1$ sont deux à deux indépendants.

```

        printf("FACE !\n");
    }
    return 0;
}

```

Question 10: Écrire la fonction

```
void graphe_alea(int n, double p);
```

qui écrit dans `mat` la matrice d'adjacence d'un tirage de $G(n, p)$.

Visualiser un tirage de $G(20, 1/4)$.

--- * ---

5 Lecture de fichiers dot

Question 11: (Bonus) Écrire la fonction

```
int lire_dot(const char *nom_fichier);
```

qui met dans `mat` le graphe décrit au format `dot` dans le fichier `nom_fichier`.

Cette fonction retourne 0 si tout s'est bien passé, -1 si le fichier n'a pu être ouvert en lecture et -2 si le nombre de sommets du graphe décrit dans ce fichier dépasse la constante `ORDRE_MAX`. On pourra utiliser les fonctions `fgets` et `fscanf` et supposer que le format du fichier respecte scrupuleusement celui donné dans le premier exemple.

--- * ---

Rappels sur les entrées-sorties standard en C

Le bout de code suivant ouvre un fichier en écriture, vérifie que cette ouverture s'est bien passée, écrit des caractères dans ce fichier et finalement le ferme (si le fichier existait déjà, il commence par être vidé).

```

#include <stdio.h> /* fopen, fclose, fprintf, perror, FILE */

int main()
{
    FILE *f;
    int i;
    char nom_fichier[] = "bouillie.txt";
    f = fopen(nom_fichier, "w");
    if (f == NULL) {
        perror("fopen"); /* écrit la dernière erreur rencontrée */
        return 1;
    }
    fprintf(f, "Je fais de la bouillie pour mes petits cochons :\n");
    for (i = 1; i < 10; i++)
        fprintf(f, "pour %d,\n", i);
}

```

```
    fprintf(f, "boeuf !!\n");  
    fclose(f);  
    return 0;  
}
```

Si vous vous sentez un peu perdu, nous vous recommandons vivement de :

- recopier **à la main** (sans copier-coller) le programme précédent ;
- de le compiler et de l'exécuter ;
- d'en visualiser le résultat (ouvrir le fichier `bouillie.txt` qui a été créé) ;
- puis de modifier ce programme (en changeant, par exemple, ce qui est affiché) et de recommencer.